Available online at www.sciencedirect.com
ScienceDirect
Computational Statistics & Data Analysis 52 (2007) 16–29
COMPUTATIONAL STATISTICS & DATA ANALYSIS
www.elsevier.com/locate/csda

Efficient algorithms for computing the best subset regression models for large-scale problems☆

Marc Hofmann[a,*], Cristian Gatu[a,d], Erricos John Kontoghiorghes[b,c]

[a] *Institut d'Informatique, Université de Neuchâtel, Switzerland*
[b] *Department of Public and Business Administration, University of Cyprus, Cyprus*
[c] *School of Computer Science and Information Systems, Birkbeck College, University of London, UK*
[d] *Faculty of Computer Science, "Alexandru Ioan Cuza" University of Iasi, Romania*

Available online 24 March 2007

Abstract

Several strategies for computing the best subset regression models are proposed. Some of the algorithms are modified versions of existing regression-tree methods, while others are new. The first algorithm selects the best subset models within a given size range. It uses a reduced search space and is found to outperform computationally the existing branch-and-bound algorithm. The properties and computational aspects of the proposed algorithm are discussed in detail. The second new algorithm preorders the variables inside the regression tree. A radius is defined in order to measure the distance of a node from the root of the tree. The algorithm applies the preordering to all nodes which have a smaller distance than a certain radius that is given a priori. An efficient method of preordering the variables is employed. The experimental results indicate that the algorithm performs best when preordering is employed on a radius of between one quarter and one third of the number of variables. The algorithm has been applied with such a radius to tackle large-scale subset-selection problems that are considered to be computationally infeasible by conventional exhaustive-selection methods. A class of new heuristic strategies is also proposed. The most important of these is one that assigns a different tolerance value to each subset model size. This strategy with different kind of tolerances is equivalent to all exhaustive and heuristic subset-selection strategies. In addition the strategy can be used to investigate submodels having noncontiguous size ranges. Its implementation provides a flexible tool for tackling large scale models.
© 2007 Elsevier B.V. All rights reserved.

Keywords: Best-subset regression; Regression tree; Branch-and-bound algorithm

1. Introduction

The problem of computing the best-subset regression models arises in statistical model selection. Most of the criteria used to evaluate the subset models rely upon the residual sum of squares (RSS) (Searle, 1971; Sen and Srivastava, 1990). Consider the standard regression model

$$y = A\beta + \varepsilon, \quad \varepsilon \sim (0, \sigma^2 I_m), \tag{1}$$

☆ The R routines can be found at URL: ⟨http://iiun.unine.ch/matrix/software⟩.

* Corresponding author. Tel.: +41 32 7182708; fax: +41 32 7182701.
E-mail addresses: marc.hofmann@unine.ch (M. Hofmann), cristian.gatu@unine.ch (C. Gatu), erricos@ucy.ac.cy (E.J. Kontoghiorghes).

0167-9473/$ - see front matter © 2007 Elsevier B.V. All rights reserved.
doi:10.1016/j.csda.2007.03.017

Table 1
Leaps and BBA: execution times in seconds for data sets of different sizes, without and with variable preordering

# Variables	36	37	38	39	40	41	42	43	44	45	46	47	48
Leaps	8	29	44	30	203	57	108	319	135	316	685	2697	6023
BBA	2	5	12	8	35	14	9	55	27	37	97	380	1722
Leaps-1	3	16	28	9	82	33	22	203	79	86	306	1326	1910
BBA-1	1	4	13	2	20	11	4	47	18	15	51	216	529

where $y \in \mathbb{R}^m$, $A \in \mathbb{R}^{m\times n}$ is the exogenous data matrix of full column rank, $\beta \in \mathbb{R}^n$ is the coefficient vector and $\varepsilon \in \mathbb{R}^n$ is the noise vector. The columns of A correspond to the exogenous variables $V = [v_1, \ldots, v_n]$. A submodel S of (1) comprises some of the variables in V. There are $2^n - 1$ possible subset models, and their computation is only feasible for small values of n. The dropping column algorithm (DCA) derives all submodels by generating a regression tree (Clarke, 1981; Gatu and Kontoghiorghes, 2003; Smith and Bremner, 1989). The parallelization of the DCA moderately improves its practical value (Gatu and Kontoghiorghes, 2003). Various procedures such as the forward, backward and stepwise selection try to identify a subset by inspecting very few combinations of variables. However, these methods rarely succeed in finding the best submodel (Hocking, 1976; Seber, 1977). Other approaches for subset-selection include ridge regression, the nonnegative garrote and the lasso (Breiman, 1995; Fan and Li, 2001; Tibshirani, 1996). Sequential replacement algorithms are fairly fast and can be used to give some indication of the maximum size of the subsets that are likely to be of interest (Hastie et al., 2001). The branch-and-bound algorithms for choosing a subset of k features from a given larger set of size n have also been investigated within the context of feature selection problems (Narendra and Fukunaga, 1997; Roberts, 1984; Somol et al., 2004). These strategies are used when the size k of the subset to be selected is known. Thus, they search over $n!/(k!(n-k)!)$ subsets.

A computationally efficient branch-and-bound algorithm (BBA) has been devised (Gatu and Kontoghiorghes, 2006; Gatu et al., 2007). The BBA avoids the computation of the whole regression tree and it derives the best subset model for each number of variables. That is, it computes

$$\underset{S}{\operatorname{argmin}}\ \mathrm{RSS}(S) \quad \text{subject to } |S| = k \ \text{ for } k = 1, \ldots, n. \tag{2}$$

The BBA was built around the fundamental property

$$\mathrm{RSS}(S_1) \geqslant \mathrm{RSS}(S_2) \quad \text{if } S_1 \subseteq S_2, \tag{3}$$

where S_1 and S_2 are two variable subsets of V (Gatu and Kontoghiorghes, 2006). The BBA-1, which is an extension of the BBA, preorders the n variables according to their strength in the root node. The variables i and j are arranged such that $\mathrm{RSS}(V_{-i}) \geqslant \mathrm{RSS}(V_{-j})$ for each $i \leqslant j$, where V_{-i} is the set V from which the ith variable has been deleted. The BBA-1 has been shown to outperform the previously introduced leaps-and-bounds algorithm (Furnival and Wilson, 1974). Table 1 shows the execution times of the BBA and leaps-and-bounds algorithm for data sets with 36–48 variables. Note that the BBA outperforms the leaps-and-bounds with preordering in the root node (Leaps-1). A heuristic version of the BBA (HBBA) that uses a tolerance parameter to relax the BBA pruning test has been discussed. The HBBA might not provide the optimal solution, but the relative residual error (RRE) of the computed solution is smaller than the tolerance employed.

Often models within a given size range must be investigated. These models, hereafter called subrange subset models, do not require the generation of the whole tree. Thus, the adaptation of the BBA for deriving the subrange subset models is expected to have a lower computational cost, and thus, it can be feasible to tackle larger scale models. The structural properties of a regression tree strategy which generates the subrange subset models is investigated and its theoretical complexity derived. A new nontrivial preordering strategy that outperforms the BBA-1 is designed and analyzed. The new strategy, which can be found to be significantly faster than existing ones, can derive the best subset models from a larger pool of variables. In addition, some new heuristic strategies based on the HBBA are developed. The tolerance parameter is either a function of the level in the regression tree, or of the size of the subset model. The novel strategies decrease execution time while selecting models of similar, or of even better, quality.

The proposed strategies, which outperform the existing subset selection BBA-1 and its heuristic version, are aimed at tackling large-scale models. The next section briefly discusses the DCA, and it introduces the all-subset-models regression tree. It generalizes the DCA so as to select only the submodels within a given size range. Section 3 discusses a novel strategy that preorders the variables of the nodes in various levels of the tree. The significant improvement in the computational efficiency when compared to the BBA-1 is illustrated. Section 4 presents and compares various new heuristic strategies. Theoretical and experimental results are presented. Conclusions and proposals for future work are discussed in Section 5.

The algorithms were implemented in C++ and are available in a package for the R statistical software environment (R Development Core Team, 2005). The GNU compiler collection was used to generate the shared libraries. The tests were run on a Pentium-class machine with 512 Mb of RAM in a Linux environment. Real and artificial data have been used in the experiments. A set of artificial variables has been randomly generated. The response variable of the true model is based on a linear combination of a subset of these artificial variables with the addition of some noise. An intercept term is included in the true model.

2. Subrange model selection

The DCA employs a straightforward approach to solve the best-subset problem (2). It enumerates and evaluates all possible $2^n - 1$ subsets of V. It generates a regression tree consisting of 2^{n-1} nodes (Gatu and Kontoghiorghes, 2003; Smith and Bremner, 1989). Each node in the tree corresponds to a subset $S = [s_1, \ldots, s_{n_s}]$ of n_s variables and to an index k ($k = 0, \ldots, n_s - 1$). The $n_s - k$ subleading models $[s_1, \ldots, s_{k+1}], \ldots, [s_1, \ldots, s_{n_s}]$ are evaluated. A new node is generated by deleting a variable. The descending nodes are given by

$$(\mathrm{drop}(S, k+1), k), (\mathrm{drop}(S, k+2), k+1), \ldots, (\mathrm{drop}(S, n_s - 1), n_s - 2).$$

Here, the operation $\mathrm{drop}(S, i)$ computes a new subset which corresponds to the subset S from which the ith variable has been deleted. This is equivalent to downdating the QR decomposition after the ith column has been deleted (Golub and Van Loan, 1996; Kontoghiorghes, 2000; Smith and Bremner, 1989). The DCA employs Givens rotations to move efficiently from one node to another.

The search space of all possible variable subset models can be reduced by imposing bounds on the size of the subset models. The subrange model selection problem is to derive:

$$S_j^* = \underset{S}{\mathrm{argmin}}\ \mathrm{RSS}(S) \quad \text{subject to } |S| = j \ \text{ for } \ j = n_a, \ldots, n_b, \tag{4}$$

where n_a and n_b are the subrange bounds ($1 \leqslant n_a \leqslant n_b \leqslant n$). The DCA and Subrange DCA (RangeDCA) are equivalent when $n_a = 1$ and $n_b = n$. The RangeDCA generates a subtree of the original regression tree. The nodes (S, k) are not computed when $n_s < n_a$ or $k \geqslant n_b$. This is illustrated in Fig. 1. The DCA regression tree with $n = 5$ variables is shown. The blank nodes represent the RangeDCA subtree for $n_a = n_b = 3$. Portions of the tree that are not computed by the RangeDCA are shaded. The nodes in the last two levels of the tree evaluate subset models of sizes 1 and 2 , i.e.

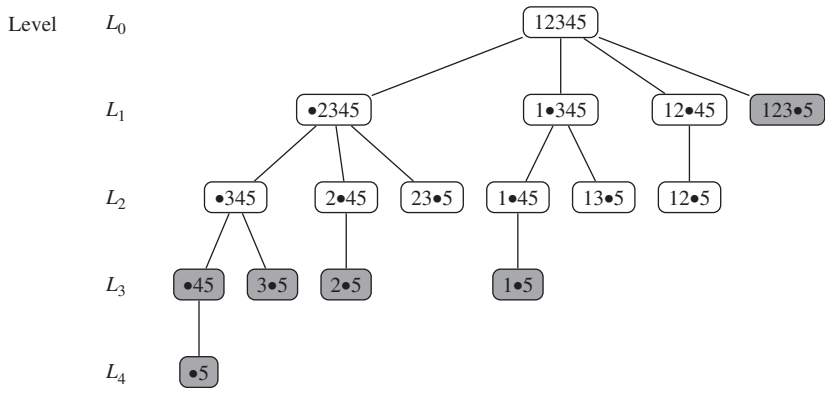


Fig. 1. The RangeDCA subtree, where $n = 5$ and $n_a = n_b = 3$.

the subsets [4], [5], [4, 5], [3, 5], [2, 5] and [1, 5]. These nodes are discarded by the RangeDCA (case $n_s < n_a$). The leftmost node in the tree evaluates the subset model [1, 2, 3, 5] of size 4 . The RangeDCA discards this node (case $k \geqslant n_b$). The Appendix provides a detailed and formal analysis of the RangeDCA.

The branch-and-bound strategy can be applied to the subtree generated by the RangeDCA. This strategy is called RangeBBA and is summarized in Algorithm 1. The RangeBBA stores the generated nodes of the regression subtree in a list. The list is managed according to a last-in, first-out (LIFO) policy. The RSSs of the best subset models are recorded in a table r. The entry r_i holds the RSS of the best current submodel of size i. The initial residuals table may be given a priori based on some earlier results, otherwise the initial residuals are set to positive infinity. The entries are sorted in decreasing order. Each iteration removes a node (S, k) from the list. The subleading model $[s_1, \ldots, s_i]$ is evaluated and compared to the entry r_i in the residuals table for $i = k+1, \ldots, n_s$. The entry r_i is updated when $\mathrm{RSS}([s_1, \ldots, s_i]) < r_i$. If $n_s \leqslant n_a$, then no child nodes are computed and the iteration terminates; otherwise, the cutting test $b_S > r_i$ is computed for $i = k+1, \ldots, \min(n_b, n_s - 1)$ and $b_S = \mathrm{RSS}(S)$. If the test fails the child node $(\mathrm{drop}(S, i), i-1)$ is generated and inserted into the node list. Note that, if $i < n_a$, then the value r_{n_a} is used in the cutting test. This is illustrated on Line 11 of the algorithm. The modified cutting test is more efficient than that of the BBA, since $r_{n_a} \leqslant r_i$ for $i = 1, \ldots, n_a - 1$. The algorithm terminates when the node list is empty. Notice that the RangeBBA with preordering (RangeBBA-1) is obtained by sorting the variables in the initial set V. The RangeBBA outperforms the standard BBA, since it uses a reduced search space and a more efficient cutting test.

Algorithm 1. The subrange BBA (RangeBBA)

```
1   procedure RangeBBA(V, n_a, n_b, r)
2       insert (V, 0) into the node list
3       while the node list is not empty do
4           Extract (S, k) from the node list
5           n_s ← |S|
6           Update the residuals r_{k+1}, ..., r_{n_s}
7           if n_s > n_a then
8               b_S ← RSS(S)
9               for i = k + 1, ..., min(n_b, n_s − 1) do
10                  j ← max(i, n_a)
11                  if b_S > r_j go to line 3
12                  S' ← drop(S, i)
13                  Insert (S', i − 1) into the node list
14              end for
15          end if
16      end while
17  end procedure
```

The effects of the subrange bounds n_a and n_b on the computational performance of the RangeDCA and RangeBBA-1 have been investigated. The Fig. 2(a) and (b) show the execution times of the RangeDCA for $n = 20$ variables and RangeBBA-1 for 36 variables, respectively. It can be observed that the RangeDCA is computationally effective in two cases: for narrow size ranges (i.e. $n_b - n_a < 2$) or extreme ranges (i.e. $n_a = 1$ and $n_b < n/4$ or $n_a > 3n/4$ and $n_b = n$). The RangeBBA-1, on the other hand, is effective for all ranges such that $n_a > n/2$. This is further confirmed by the results in Table 2. The number of nodes generated by the RangeBBA-1 for the 15 variable POLLUTE data set (Miller, 2002) is shown. All possible subranges $1 \leqslant n_a \leqslant n_b \leqslant 15$ are considered. For the case $n_a = 1$ and $n_b = 15$ the RangeBBA-1 generates 381 nodes and is equivalent to the BBA-1 (Gatu and Kontoghiorghes, 2006).

3. Radius preordering

The BBA with an initial preordering of the variables in the root node (BBA-1) significantly increases the computational speed. The cost of preordering the variables once is negligible. The aim is to consider a strategy that applies the preordering of variable subsets inside the regression tree and that yields a better computational performance than

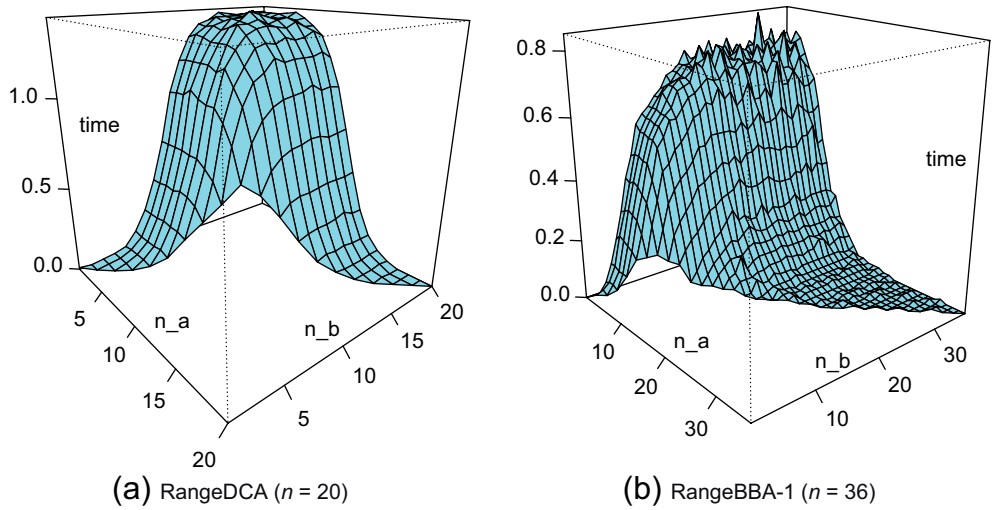


Fig. 2. Subrange model selection: execution times in seconds for varying n_a and n_b.

Table 2
Number of nodes generated by the RangeBBA-1 to compute the best subset models of the POLLUTE data set for different size ranges

n_a	n_b														
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	12	37	96	178	276	332	356	373	375	376	377	378	380	381	381
2		31	90	172	270	326	350	367	369	370	371	372	374	375	375
3			75	157	255	311	335	352	354	355	356	357	359	360	360
4				123	221	277	301	318	320	321	322	323	325	326	326
5					173	229	253	270	272	273	274	275	277	278	278
6						103	127	144	146	147	148	149	151	152	152
7							52	69	71	72	73	74	76	77	77
8								38	40	41	42	43	45	46	46
9									11	12	13	14	16	17	17
10										11	12	13	15	16	16
11											12	13	15	16	16
12												13	15	16	16
13													15	16	16
14														15	15
15															1

the BBA-1. The new strategy is hereafter called radius preordering BBA (RadiusBBA). The RadiusBBA sorts the variables according to their strength. The strength of the ith variable is given by its bound $\mathrm{RSS}(S_{-i}) = \mathrm{RSS}(\mathrm{drop}(S, i))$. The main tool for deriving the bound is the downdating of the QR decomposition after the corresponding column of the data matrix has been deleted. This has a cost, and therefore care must be taken to apply a preordering to the nodes whenever the expected gain outweighs the total cost inherent to the preordering process.

The RadiusBBA preorders the variables at the roots of large subtrees. The size of the subtree with root (S, k) is given by 2^{d-1}, where n_s is the number of variables in S and $d = n_s - k$ is the number of active variables. The RadiusBBA defines the node radius $\rho = n - d$, where n is the initial number of variables in the root node $(V, 0)$. The radius of a node is a measure of its distance from the root node. Notice that the root nodes of larger subtrees have a smaller radius, while the roots of subtrees with the same number of nodes have the same radius. Given a radius P

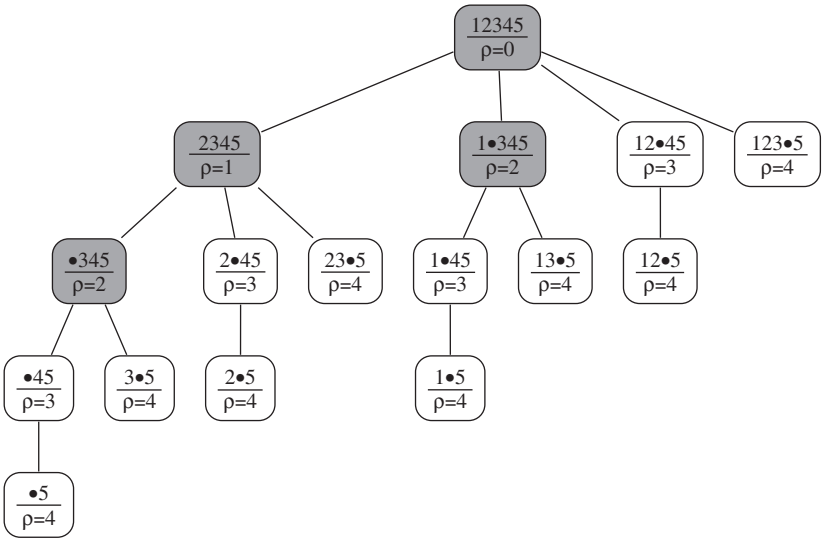


Fig. 3. Nodes that apply preordering for radius $P = 3$.

the preordering of variables is applied only to nodes of radius $\rho < P$, where $0 \leqslant P \leqslant n$. If $P = 0$ and $P = 1$, then the RadiusBBA is equivalent to the BBA and BBA-1, respectively. If $P = n$, then the active variables are preordered in all nodes. Fig. 3 illustrates the radius of every node in a regression tree for $n = 5$ variables. Shaded nodes are preordered by the RadiusBBA, where $P = 3$.

The RadiusBBA is illustrated in Algorithm 2. A node is extracted from the node list at each iteration. If the node radius ρ is less than the given preordering radius P, then the active variables are preordered before updating the residuals table. The nodes which cannot improve the current best solutions are not generated. The cutting test (see line 11) compares the bound of the current node to the corresponding entry in the residuals table in order to decide whether or not to generate a child node.

Algorithm 2. The radius brnach-and-bound algorithm (RadiusBBA)

```
1   procedure RadiusBBA(V, P, r)
2       n ← |V|
3       Insert (V, 0) into node list
4       while node list is not empty do
5           Extract (S, k) from nodes list
6           n_s ← |S|; ρ ← n − k
7           if ρ < P then preorder [s_{k+1}, ..., s_{n_s}]
8           Update residuals r_{k+1}, ..., r_{n_s}
9           b_S ← RSS(S)
10          for i = k + 1, ..., n_s − 1 do
11              if b_S > r_i goto line 3
12              S′ ← drop(S, i)
13              Insert (S′, i − 1) into node list
14          end for
15      end while
16  end procedure
```

The preordering process sorts the variables in order of decreasing bounds. Given a node (S, k), the bound of the ith active variable $(i = 1, \ldots, d)$ is $\mathrm{RSS}(\mathrm{drop}(S, k + i))$, i.e. the RSS of the model from which the $(k + i)$th variable has been removed. The d active variables of the node (S, k) are represented by the leading $d \times d$ submatrix of the upper triangular $R \in \mathbb{R}^{(d+1)\times(d+1)}$ factor of the QR decomposition. The last column of R corresponds to the response

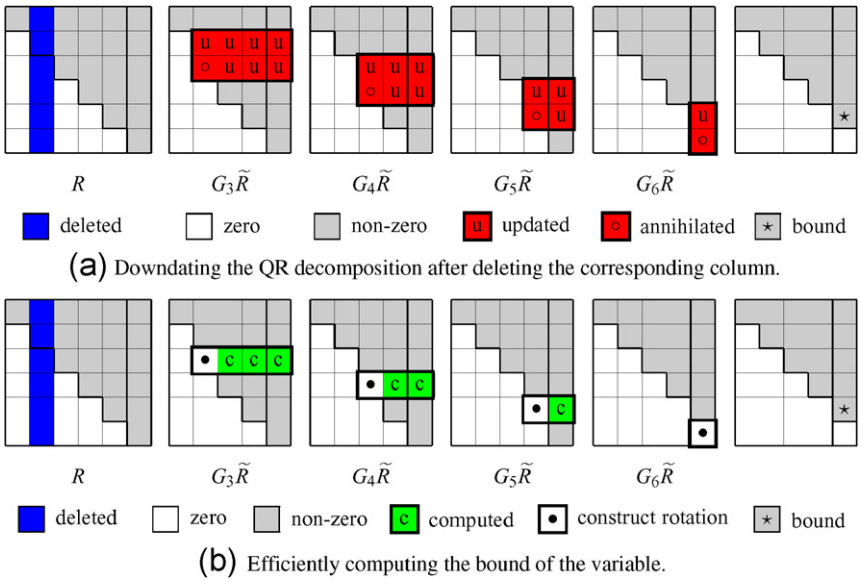


Fig. 4. Exploiting the QR decomposition to compute the bound of the ith active variable, when $i = 2$.

variable y in (1). Let $\tilde{R}$ denote R without its ith column. The drop operation applies $d - i + 1$ biplanar Givens rotations to retriangularize $\tilde{R}$. That is, it computes

$$G_d \cdots G_i \tilde{R} = \begin{pmatrix} \hat{R} \\ 0 \end{pmatrix} \begin{matrix} d \\ 1 \end{matrix}, \tag{5}$$

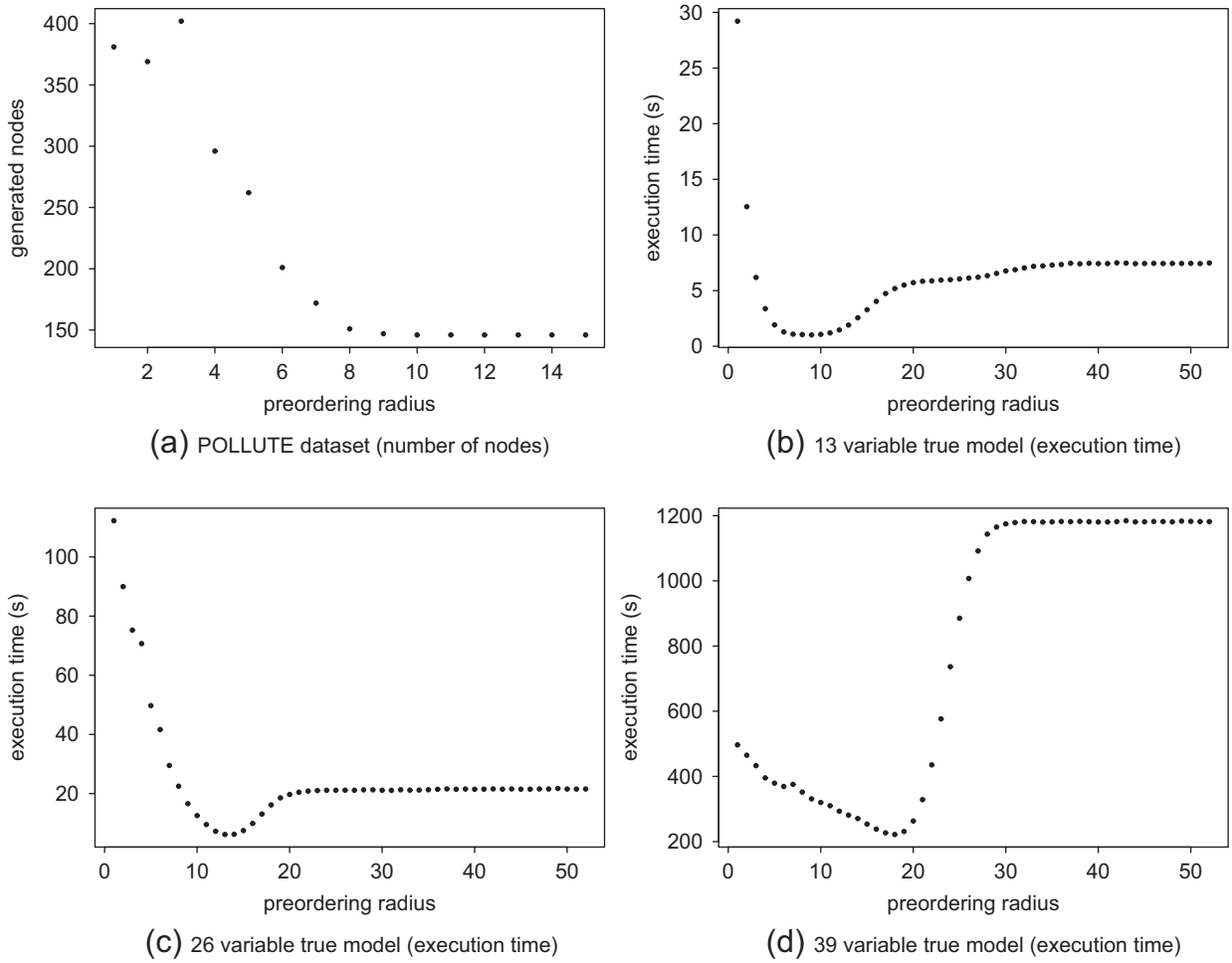
where $\hat{R} \in \mathbb{R}^{d \times d}$ is upper triangular. The bound of the ith active variable, that is, the RSS of the model after deleting the ith variable, is given by $\hat{R}^2_{d,d}$—the square of the diagonal element of $\hat{R}$ in position (d, d).

Note that the rotation $G_j \tilde{R}$ $(j = i, \ldots, d)$ affects only the last $(d - j + 1)$ elements of rows j and $j + 1$ of $\tilde{R}$, and reduces to zero $\tilde{R}_{j+1,j}$. The application of a rotation to two biplanar elements x and y can be written as

$$\begin{pmatrix} \tilde{x} \\ \tilde{y} \end{pmatrix} = \begin{pmatrix} c & s \\ -s & c \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}.$$

If c and s are chosen such that $c = x/t$ and $s = y/t$, then $\tilde{x} = t$ and $\tilde{y} = 0$, where $t^2 = x^2 + y^2$ and $t \neq 0$. The number of nodes in which the variables are preordered increases exponentially with the preordering radius P. This computational overhead will have a significant impact on the overall performance of the RadiusBBA. Fig. 4(a) shows the retriangularization of a 6×6 triangular matrix after deleting the second column using Givens rotations.

The complete and explicit triangularization of $\tilde{R}$ should be avoided and only the bound of the deleted ith variable, i.e. $\hat{R}^2_{d,d}$, should be computed. This can be achieved by computing only the elements of $\tilde{R}$ which are needed in deriving $\hat{R}_{d,d}$. Thus, the Givens rotation $G_j \tilde{R}$ $(j = i, \ldots, d + 1)$ explicitly updates only the last $(d - j + 1)$ elements of the $(j + 1)$th row of $\tilde{R}$ which are required by the subsequent rotation. The jth row of $\tilde{R}$ is not updated and neither is the subdiagonal element $\tilde{R}_{j+1,j}$ annihilated. This strategy has approximately half the cost for deriving the bound of the ith variable. In order to optimize the implementation of this procedure, the original triangular R is not modified and the bounds are computed without copying the matrix to temporary store. Fig. 4(b) illustrates the steps of this strategy, while Algorithm 3 shows the exact computational steps.



(a) POLLUTE dataset (number of nodes) (b) 13 variable true model (execution time)

(c) 26 variable true model (execution time) (d) 39 variable true model (execution time)

Fig. 5. Computational cost of RadiusBBA for the POLLUTE data set and artificial data with 52 variables.

Algorithm 3. Computing the bound of the ith active variable

procedure bound(R, i, b)
  $x_j \leftarrow R_{i+1,j}$, for $j = i+1, \ldots, d+1$
  for $j = i+1, \ldots, d+1$ **do**
    $y_k \leftarrow R_{j+1,k}, k = j, \ldots, d+1$
    $t \leftarrow \sqrt{x_j^2 + y_j^2}; c \leftarrow x_j/t; s \leftarrow y_j/t$
    $x_k \leftarrow -s \cdot x_k + c \cdot y_k, k = j+1, \ldots, d+1$
  end for
  $b \leftarrow t^2$
end procedure

Fig. 5 illustrates the effect of the preordering radius P on the RadiusBBA. Fig. 5(a) illustrates the number of nodes generated by the RadiusBBA on the POLLUTE data set (15 variables) for every preordering radius ($1 \leqslant P \leqslant 15$). The BBA-1 generates 318 nodes. The number of nodes generated by the RadiusBBA decreases steadily up to $P = 8$ where a minimum of 146 nodes is reached. The other three figures illustrate the execution times of the RadiusBBA on artificial data sets with 52 variables. The true model comprises 13, 26 and 39 variables, respectively. In all three cases the RadiusBBA represents a significant improvement over the BBA-1. In the case of the small true model (13 variables),

Table 3
Execution times in seconds of the subrange RadiusBBA with radius 21 and 26 on data sets comprising 64 and 80 variables, respectively

n_{true}	n_a	n_b	Time
$n = 64$			
16	1	64	119
	8	24	50
32	1	64	3415
	24	40	4
48	1	64	3531
	36	60	1
$n = 80$			
20	1	80	4205 (70 min)
	10	30	2309 (38 min)
40	1	80	177383 (2 days)
	20	60	25732 (8 h)
60	1	80	1293648 (15 days)
	40	80	178 (3 min)

The true model comprises n_{true} variables.

the BBA-1 and the RadiusBBA with $P = 9$ require 30 and 1 s, respectively. In the case of a medium size true model (26 variables), the time is reduced from 112 to 6 s, i.e. the RadiusBBA with radius 13 is almost 20 times faster. In the third case (big true model with 39 variables) the RadiusBBA with radius 18 is over 2 times faster than the BBA-1 which requires 500 s. These tests show empirically that values of P lying between $n/4$ and $n/3$ are a good choice for the RadiusBBA.

Table 3 shows the execution times of the RadiusBBA on two data sets with $n = 64$ and 80 variables, respectively. The preordering radius used is $P = \lfloor n/3 \rfloor$. The number of variables in the true model is given by n_{true}. Different ranges n_a and n_b have been used. For the full range, i.e. $n_a = 1$ and $n_b = n$, the RadiusBBA computes the best subset models for model sizes which are computationally infeasible for the BBA-1. It can be observed that the use of smaller ranges significantly reduces the time required by the RadiusBBA for deriving the best subset models.

4. Heuristic strategies

The Heuristic BBA (HBBA) relaxes the objective of finding an optimal solution in order to gain in computational efficiency. That is, the HBBA is able to tackle large-scale models when the exhaustive BBA is found to be computationally infeasible. The heuristic algorithm ensures that

$$\mathrm{RRE}(\tilde{S}_i) < \tau \quad \text{for } i = 1, \ldots, n, \tag{6}$$

where $\widetilde{S}_i$ is the (heuristic) solution subset model of size i and τ is a tolerance parameter ($\tau > 0$). Generally, the RRE of a subset S_i is given by

$$\mathrm{RRE}(\mathrm{S}_i) = \frac{|\mathrm{RSS}(\mathrm{S}_i) - \mathrm{RSS}(\mathrm{S}_i^*)|}{\mathrm{RSS}(\mathrm{S}_i^*)},$$

where S_i^* is the optimal subset of size i reported by the BBA. The space of all possible submodels is not searched exhaustively. The HBBA aims to find an acceptable compromise between the brevity of the search ($\tau \to \infty$) and the quality of the solutions computed ($\tau \to 0$). The modified cutting test in Gatu and Kontoghiorghes (2006) is given by

$$(1 + \tau) \cdot \mathrm{RSS}(S) > r_{j+1}. \tag{7}$$

Note that the HBBA is equivalent to the BBA for $\tau = 0$. Furthermore, the HBBA reduces to the DCA if $\tau = -1$. Notice that in (7) $r_{j+1} < 0$ for $\tau = -1$, which implies that the cutting test never holds and all the nodes of the tree are generated.

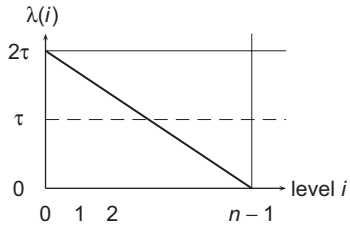


Fig. 6. The $\lambda(i)$ level tolerance function.

Table 4
Mean number of nodes and RREs generated by the HBBA and LevelHBBA

n_{true}	9		18		27	
Algorithm	Nodes	RRE	Nodes	RRE	Nodes	RRE
HBBA	14 278	6e − 4	47 688	3e − 4	35 062	9e − 4
LevelHBBA	13 129	8e − 4	34 427	5e − 4	21 455	3e − 3

In order to increase the capability of the heuristic strategy to tackle larger subset-selection problems, a new heuristic algorithm is proposed. The Level HBBA (LevelHBBA) employs different values of the tolerance parameter in different levels of the regression tree. It uses higher values in the levels close to the root node to encourage the cutting of large subtrees. Lower tolerance values are employed in lower levels of the tree in order to select good quality subset models. The indices of the tree levels are shown in Fig. 1. The tolerance function employed by the LevelHBBA is defined formally as

$$\lambda(i) = 2\tau(n - i - 1)/(n - 1) \quad \text{for } i = 0, \ldots, n - 1,$$

where i denotes the level of the regression tree and τ the average tolerance. The graph of the function $\lambda(i)$ is shown in Fig. 6.

The HBBA and LevelHBBA were executed on data sets with 36 variables. Three types of data sets were employed, with a small, a medium and a big true model comprising 9, 18 and 27 variables, respectively. The tolerance parameter τ has been set to 0.2. The results are summarized in Table 4. The table shows the number of nodes and the mean RRE. Each experiment has been repeated 32 times. The values shown in the table are overall means. The LevelHBBA generates slightly fewer nodes, but it produces results that are of lesser quality than those computed by the HBBA. Notice that the average RRE is significantly lower than the tolerance employed.

The Size HBBA (SizeHBBA) assigns a different tolerance value to each subset model size. It can be seen as a generalization of the HBBA and the RangeBBA. The degree of importance of each subset size can be expressed. Lower tolerance values are attributed to subset sizes of greater importance. Less relevant subset sizes are given higher tolerance values. Subset model sizes can be effectively excluded from the search by setting a very high tolerance value. Thus, unlike the RangeBBA, the SizeHBBA can be employed to investigate non-contiguous size ranges. The SizeHBBA satisfies

$$\text{RRE}(\tilde{S}_i) \leqslant \sigma_i \quad \text{for } i = 1, \ldots n,$$

where i denotes the size of the subset model and σ_i the corresponding tolerance value. Given a node (S, k), the child node $(\text{drop}(S, j), j - 1)$ is cut if:

$$(1 + \sigma_i) \cdot \text{RSS}(S) > r_i \quad \text{for } i = j, \ldots, n_s - 1.$$

Table 5
Mean number of nodes and RREs generated by the HBBA and SizeHBBA

n_{true}	9		18		27	
Algorithm	Nodes	RRE	Nodes	RRE	Nodes	RRE
HBBA	12 781	8e − 4	38 716	4e − 4	39 907	1e − 3
SizeHBBA	15 079	2e − 4	39 457	2e − 4	40 250	3e − 4

The SizeHBBA generalizes the previous algorithms, i.e.,

$$\text{SizeHBBA} \equiv \begin{cases} \text{DCA} & \text{if } \sigma_i = -1, \\ \text{RangeDCA} & \text{if } \sigma_i = -1 \text{ for } n_a \leqslant i \leqslant n_b \text{ and } \sigma_i \gg 0 \text{ otherwise}, \\ \text{BBA} & \text{if } \sigma_i = 0, \\ \text{RangeBBA} & \text{if } \sigma_i = 0 \text{ for } n_a \leqslant i \leqslant n_b \text{ and } \sigma_i \gg 0 \text{ otherwise}, \\ \text{HBBA} & \text{if } \sigma_i = \tau. \end{cases}$$

The SizeHBBA is equivalent to all previously proposed algorithms with the exception of the LevelHBBA. Thus, it can be seen as more than a mere heuristic algorithm and allows a very flexible investigation of all subset models.

It has been observed experimentally that the SizeHBBA is efficient compared to the HBBA when a tolerance value τ is used for the first half of model sizes and zero-tolerance for the remaining sizes. That is, when the optimal solution is guaranteed to be found for submodel sizes between $n/2$ and n. Table 5 shows the computational performance of the HBBA and SizeHBBA on data sets with 36 variables. The HBBA is executed with $\tau = 0.2$, while the SizeHBBA is executed with $\sigma_i = \tau$ for $i \leqslant 18$ and $\sigma_i = 0$ otherwise. The results show that, without a significant increase in computational cost, there is a gain in solution quality, i.e. the optimal subset models with 18 or more variables are found. Furthermore, the results are consistent with the observed behavior of the RangeBBA. For bigger models, larger subranges can be chosen at a reasonable computational cost. In case of the SizeHBBA, constraints on larger submodels can be stricter (i.e. lower tolerance) without additional computational cost. This may be due to the asymmetric structure of the tree, i.e. subtrees are smaller on the right hand side.

5. Conclusions

Various algorithms for computing the best subset regression models have been developed. They improve and extend previously introduced exhaustive and heuristic strategies which were aimed at solving large-scale model-selection problems. The proposed algorithms are based on a dropping column algorithm (DCA) which derives all possible subset models by generating a regression tree (Gatu and Kontoghiorghes, 2003, 2006; Smith and Bremner, 1989).

An algorithm (RangeDCA) that computes the all-subsets models within a given range of model sizes has been proposed. The RangeDCA is a generalization of the DCA and it generates only a subtree of the all-subsets tree derived by the DCA. Theoretical measures of complexity of the RangeDCA have been derived and analyzed (see Appendix). The theoretical complexities have been confirmed through experiments. The branch-and-bound strategy in Gatu and Kontoghiorghes (2006) has been applied in the tree that is generated by the RangeDCA.

The preordering of the initial variable set (BBA-1) significantly improves the computational performance of the BBA. However, the BBA-1 might fail to detect significant combinations of variables in the root node. Hence, a more robust preordering strategy is designed. Subsets of variables are sorted inside the regression tree after some variables have been deleted. Thus, important combinations of variables are more likely to be identified and exploited by the algorithm.

A preordering BBA (RadiusBBA) which generalizes the BBA-1 has been designed. The RadiusBBA applies variable preordering to nodes of arbitrary radius in the regression tree rather than to the root only. The radius provides a measure of the distance between a node and the root. Experiments have shown that the number of nodes computed by the RadiusBBA decreases as the preordering radius increases. However, the preordering requires the retriangularization of an upper triangular matrix after deleting a column and it incurs a considerable computational overhead. A computationally efficient strategy has been designed, which avoids the explicit retriangularization used to compute the strength of a variable. This reduces the total overhead of the RadiusBBA. In various experiments, it has been observed

that the best performance is achieved when preordering is employed with a radius of between one quarter and one third of the number of variables. The RadiusBBA significantly reduces the computational time required to derive the best submodels when compared to the existing BBA-1. This allows the RadiusBBA to tackle subset-selection problems that have previously been considered as computationally infeasible.

A second class of algorithms has been designed, which improve the heuristic version of the BBA (HBBA) (Gatu and Kontoghiorghes, 2006). The Level HBBA (LevelHBBA) applies different tolerances on different levels of the regression tree. The LevelHBBA generates fewer nodes than the HBBA when both algorithms are applied with the same mean tolerance. Although the subset models computed by the LevelHBBA are of lesser quality than those computed by the HBBA, the relative residual errors remain far below the mean tolerance. The size-heuristic BBA (SizeHBBA) assigns different tolerance values to subset models of different sizes. The subset models computed by the SizeHBBA improve the quality of the models derived by the HBBA. Thus, for approximately the same computational effort, the SizeHBBA produces submodels closer to the optimal ones than does the HBBA. The SizeHBBA for different kind of tolerances is equivalent to the DCA, RangeDCA, BBA, RangeBBA and HBBA. This makes the SizeHBBA a powerful and flexible tool for computing subset models. Within this context, it extends the RangeBBA by allowing the investigation of submodels of non-contiguous size ranges.

The employment by the RadiusBBA of computationally less expensive criteria in preordering the variables should be investigated. This should include the use of parallel strategies to compute the bound of the model after deleting a variable (Hofmann and Kontoghiorghes, 2006). It might be fruitful to explore the possibility of designing a dynamic heuristic BBA which automatically determines the tolerance value in a given node based on a learning strategy. A parallelization of the BBA, employing a task-farming strategy on heterogeneous parallel systems, could be considered. The adaptation of the strategies to the vector autoregressive model is currently under investigation (Gatu and Kontoghiorghes, 2005, 2006).

Acknowledgments

The authors are grateful to the guest-editor Manfred Gilli and the two anonymous referees for their valuable comments and suggestions. This work is in part supported by the Swiss National Science Foundation Grants 101412-105978, 200020-100116/1, PIOI1-110144 and PIOI1-115431/1, and the Cyprus Research Promotion Foundation Grant KY-IT/0906/09.

Appendix A. Subrange model selection: complexity analysis

Let the pair (S, k) denote a node of the regression tree, where S is a set of n variables and k the number of passive variables ($0 \leqslant k < n$). A formal representation of the DCA regression tree is given by

$$\Delta(S,k) = \begin{cases} (S,k) & \text{if } k = n-1, \\ ((S,k), \Delta(\text{drop}(S,k+1),k), \ldots, \Delta(\text{drop}(S,n-1),n-2)) & \text{if } k < n-1. \end{cases}$$

The operation $\text{drop}(S, i)$ deletes the ith variable in $S = [s_1, \ldots, s_n]$. The QR decomposition is downdated after the corresponding column of the data matrix has been deleted. Orthogonal Givens rotations are employed in reconstructing the upper-triangular factor. An elementary operation is defined as the rotation of two vector elements. The cost of one elementary operation is approximately six flops. The number of elementary operations required by the drop operation is

$$T_{\text{drop}}(S,i) = (n-i+1)(n-i+2)/2.$$

The passive variables $s_1, \ldots, s_k$ are not dropped, i.e. they are inherited by all child nodes. All active variables $s_{k+1}, \ldots, s_n$, except the last one, are dropped in turn to generate new nodes. The structure of the tree can be expressed in terms of the number of active variables $d = n - k$. This simplified representation $\Delta(d)$ of the regression tree $\Delta(S, k)$ is given by

$$\Delta(d) = \begin{cases} (d) & \text{if } d = 1, \\ ((d), \Delta(d-1), \ldots, \Delta(1)) & \text{if } d > 1, \end{cases}$$

where (d) is a node with d active variables. The number of nodes and elementary operations are calculated, respectively, by

$$N(d) = 1 + \sum_{i=1}^{d-1} N(d-i) = 2^{d-1}$$

and

$$T(d) = \sum_{i=1}^{d-1} (T_{\text{drop}}(d, i) + T(d-i)) = 7 \cdot 2^{d-1} - (d^2 + 5d + 8)/2.$$

Here, $T_{\text{drop}}(d, i)$ is the complexity of dropping the ith of d active variables ($i = 1, \ldots, d$).

Let n_a designate a model size ($1 \leqslant n_a \leqslant n$). Then, $\Delta_{n_a}(S, k)$ denotes the subtree of $\Delta(S, k)$ which consists of all nodes which evaluate exactly one model of size n_a ($0 \leqslant k < n_a$). It is equivalent to:

$$\Delta_a(d) = \begin{cases} (d) & \text{if } d = a, \\ ((d), \Delta_a(d-1), \ldots, \Delta_1(d-a)) & \text{if } d > a, \end{cases}$$

where $a = n_a - k$. The number of nodes is calculated by

$$\begin{aligned} N_a(d) &= \begin{cases} 1 & \text{if } d = a, \\ 1 + \sum_{i=1}^{a} N_{a-i+1}(d-i) & \text{if } d > a \end{cases} \\ &= C_a^d = \frac{d!}{a!(d-a)!}. \end{aligned}$$

Similarly, the number of elementary operations required to construct $\Delta_a(d)$ is calculated by

$$\begin{aligned} T_a(d) &= \begin{cases} 0 & \text{if } d = a, \\ \sum_{i=1}^{a} (T_{\text{drop}}(d, i) + T_{a-i+1}(d-i)) & \text{if } d > a \end{cases} \\ &= T_{\text{drop}}(d, 1) + T_{a-1}(d-1) + T_a(d-1). \end{aligned}$$

The closed form

$$T_a(d) = \sum_{i=0}^{a-1} \sum_{j=i}^{d-a+i-1} C_i^j \cdot T_{\text{drop}}(d-j, 1)$$

is obtained through the generating function

$$G(x, y) = (1 - y(1+x))^{-1} \sum_{0<i<j} T_{\text{drop}}(j, i) x^i y^j$$

of $T_a(d)$, where $k = 0$, $a = n_a$ and $d = n$. That is, this is the number of elementary operations necessary to compute all subset models comprising n_a out of n variables.

Now, let $\Delta_{n_a, n_b}(S, k)$ denote the tree which evaluates all subset models with more than n_a and less than n_b variables, inclusively ($1 \leqslant n_a \leqslant n_b \leqslant n$ and $0 \leqslant k < n_a$). It is equivalent to

$$\Delta_{a,b}(d) = \begin{cases} (d) & \text{if } d = a, \\ \Delta_{a,b-1}(d) & \text{if } d = b, \\ ((d), \Delta_{a,b}(d-1), \ldots, \Delta_{1,b-a+1}(d-a), \ldots, \Delta_{1,1}(d-b)) & \text{if } d > b, \end{cases}$$

where $a = n_a - k$ and $b = n_b - k$. This tree can be seen as the union of all trees $\Delta_c(d)$, for $c = a, \ldots, b$. Hence, the number of nodes and operations can be calculated, respectively, by

$$N_{a,b}(d) = \sum_{c=a}^{b} N_c(d) - \sum_{c=a}^{b-1} N'_c(d)$$

and

$$T_{a,b}(d) = \sum_{c=a}^{b} T_c(d) - \sum_{c=a}^{b-1} T'_c(d).$$

Now,

$$N'_c(d) = C_c^{d-1}$$

and

$$T'_c(d) = \sum_{i=0}^{c-1} \sum_{j=i}^{d-c+i-2} C_i^j \cdot T_{\text{drop}}(d-j, 1)$$

are the nodes and operations which have been counted twice. Specifically, these are given by the subtrees $\Delta'_{n_c}(S, k)$ which represent the intersection of the two trees $\Delta_{n_c}(S, k)$ and $\Delta_{n_c+1}(S, k)$ for $1 \leqslant n_c < n$. Their structure is given by

$$\Delta'_c(d) = \begin{cases} (d) & \text{if } d = c+1, \\ ((d), \Delta'_c(d-1), \ldots, \Delta'_1(d-a)) & \text{if } d < c+1, \end{cases}$$

where $c = n_c - k$.

References

Breiman, L., 1995. Better subset regression using the nonnegative garrote. Technometrics 37, 373–384.

Clarke, M.R.B., 1981. Statistical algorithms: algorithm AS 163: a Givens algorithm for moving from one linear model to another without going back to the data. J. Roy. Statist. Soc. Ser. C Appl. Statist. 30, 198–203.

Fan, J., Li, R., 2001. Variable selection via nonconcave penalized likelihood and its oracle properties. J. Amer. Statist. Assoc. 96, 1348–1360.

Furnival, G., Wilson, R., 1974. Regression by leaps and bounds. Technometrics 16, 499–511.

Gatu, C., Kontoghiorghes, E.J., 2003. Parallel algorithms for computing all possible subset regression models using the QR decomposition. Parallel Comput. 29, 505–521.

Gatu, C., Kontoghiorghes, E.J., 2005. Efficient strategies for deriving the subset VAR models. Comput. Manage. Sci. 2, 253–278.

Gatu, C., Kontoghiorghes, E.J., 2006. Branch-and-bound algorithms for computing the best subset regression models. J. Comput. Graph. Statist. 15, 139–156.

Gatu, C., Yanev, P., Kontoghiorghes, E.J., 2007. A graph approach to generate all possible regression submodels. Comput. Statist. Data Anal. in press, doi: 10.1016/j.csda.2007.02.018.

Golub, G.H., Van Loan, C.F., 1996. Matrix Computations. Johns Hopkins Studies in the Mathematical Sciences. third ed. Johns Hopkins University Press, Baltimore, MA.

Hastie, T., Tibshirani, R., Friedman, J., 2001. The Elements of Statistical Learning. Springer Series in Statistics. Springer, New York.

Hocking, R.R., 1976. The analysis and selection of variables in linear regression. Biometrics 32, 1–49.

Hofmann, M., Kontoghiorghes, E.J., 2006. Pipeline givens sequences for computing the QR decomposition on a EREW PRAM. Parallel Comput. 32, 222–230.

Kontoghiorghes, E.J., 2000. Parallel Algorithms for Linear Models: Numerical Methods and Estimation Problems, Advances in Computational Economics, vol. 15. Kluwer Academic Publishers, Boston.

Miller, A.J., 2002. Subset Selection in Regression Monographs on Statistics and Applied Probability, vol. 95, second ed. Chapman & Hall, London (Related software can be found at URL: ⟨http://users.bigpond.net.au/amiller/⟩).

Narendra, P.M., Fukunaga, K., 1997. A branch and bound algorithm for feature subset selection. IEEE Trans. Comput. 26, 917–922.

R Development Core Team, 2005. R: A Language and Environment for Statistical Computing. R Foundation for Statistical Computing, Vienna, Austria.

Roberts, S.J., 1984. Statistical algorithms: algorithm AS 199: a branch and bound algorithm for determining the optimal feature subset of given size. Appl. Statist. 33, 236–241.

Searle, S.R., 1971. Linear Models. Wiley, New York.

Seber, G.A.F., 1977. Linear Regression Analysis. Wiley, New York.

Sen, A., Srivastava, M., 1990. Regression Analysis. Theory, Methods and Applications. Springer, Berlin.

Smith, D.M., Bremner, J.M., 1989. All possible subset regressions using the QR decomposition. Comput. Statist. Data Anal. 7, 217–235.

Somol, P., Pudil, P., Kittler, J., 2004. Fast branch & bound algorithms for optimal feature selection. IEEE Trans. Pattern Anal. Mach. Intell. 26, 900–912.

Tibshirani, R.J., 1996. Regression shrinkage and selection via the lasso. J. Roy. Statist. Soc. Ser. B (Statist. Methodol.) 58, 267–288.